

AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 01.12.2026

Lab 07

Funkcje 

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć funkcję jako **abstrakcję**: nazwany, wielokrotnego użytku fragment algorytmu
- Parametry, argumenty, `return`, wartości domyślne, zasięg zmiennych
- Funkcje jako obiekty, `lambda`, **rekurencja**
- Pierwszy **blok pracy z AI** 🤖: wyjaśnianie i refaktoryzacja kodu
- Efekty z sylabusa: **U1, U2, U3, KS1**

CZĘŚĆ I

Teoria: funkcje

Po co funkcje?

- **DRY** (*Don't Repeat Yourself*): jedna definicja zamiast kopiowania kodu
- **Abstrakcja**: używasz `sorted()` bez wiedzy, jak działa w środku
- **Testowalność**: funkcję można sprawdzić w izolacji – tak działają nasze testy
- **Czytelność**: nazwa funkcji mówi, *co* robi kod

```
def bmi(weight, height):  
    """Zwraca wskaźnik BMI dla masy w kg i wzrostu w m."""  
    return weight / height ** 2  
  
print(f"{bmi(70, 1.75):.2f}")    # 22.86
```

Anatomia funkcji

```
def count_passing(grades, threshold=3.0):    # nagłówek: nazwa + parametry
    """Ile ocen jest >= threshold."""        # docstring (PEP 257)
    count = 0                                # zmienna lokalna
    for _, grade in grades:
        if grade >= threshold:
            count += 1
    return count                               # wynik
```

- **Parametr** – nazwa w definicji; **argument** – wartość w wywołaniu
- Funkcja bez `return` zwraca `None`
- `return` kończy funkcję natychmiast

Sposoby przekazywania argumentów

```
count_passing(grades)           # domyślne threshold=3.0
count_passing(grades, 4.5)       # pozycyjnie
count_passing(grades, threshold=4.5) # nazwanie - czytelniej
```

Pułapka: wartość domyślna liczona jest **raz**, przy definicji funkcji.

```
def add_item(item, basket=[]):    # ŹLE - jedna lista dla wszystkich wywołań
    basket.append(item)
    return basket
```

```
add_item("mleko"); add_item("chleb")    # ['mleko', 'chleb'] !
```

```
def add_item(item, basket=None):    # DOBRZE
    if basket is None:
```

```
        basket = []
```

Zasięg zmiennych: reguła LEGB

LITERA	ZASIĘG	PRZYKŁAD
L	lokalny (<i>Local</i>)	zmienne w funkcji
E	otaczający (<i>Enclosing</i>)	funkcja zewnętrzna
G	globalny (<i>Global</i>)	zmienne modułu
B	wbudowany (<i>Built-in</i>)	<code>print</code> , <code>len</code>

```
def increment():  
    global counter    # bez tego: UnboundLocalError  
    counter += 1      # działa, ale utrudnia testy – lepiej przekazać i zwrócić
```

Funkcje są obiektami

```
OPERATIONS = {"+": add, "-": subtract, "*": multiply, "/": divide}

func = OPERATIONS["*"]    # bez nawiasów – sama funkcja, nie wynik
func(6, 7)                 # 42
```

- Funkcję można zapisać w zmiennej, słowniku, przekazać jako argument
- **lambda** – krótka funkcja anonimowa (jedno wyrażenie):

```
sorted(names, key=lambda name: name.split()[-1])    # po nazwisku
list(filter(lambda n: n % 2 == 0, numbers))          # parzyste
```

Rekurencja

Definicja. Funkcja rekurencyjna wywołuje samą siebie dla **mniejszego** problemu, aż dojdzie do **przypadku bazowego**.

```
def factorial(n):  
    if n == 0:                # przypadek bazowy – warunek stopu  
        return 1  
    return n * factorial(n - 1)
```

- $\text{factorial}(3) = 3 \cdot \text{factorial}(2) = 3 \cdot 2 \cdot \text{factorial}(1) = 3 \cdot 2 \cdot 1 \cdot \text{factorial}(0) = 6$
- Każde wywołanie zajmuje miejsce na **stosie wywołań**; limit w CPythonie:
`sys.getrecursionlimit()` → 1000, potem `RecursionError`

Koszt rekurencji: Fibonacci

```
def fibonacci(n):  
    if n < 2:  
        return n  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

- `fibonacci(35)` wywołuje funkcję prawie **30 milionów** razy – te same wartości liczone są wielokrotnie
- Rozwiązanie: **zapamiętywanie** wyników (*memoizacja*) albo zwykła pętla

```
from functools import cache  
  
@cache  
def fibonacci(n): ...
```

```
if __name__ == "__main__":
```

```
def main():  
    a = float(input("Liczba: "))  
    print(calculate(a, "+", 1))  
  
if __name__ == "__main__":  
    main()
```

- `python plik.py` → `__name__ == "__main__"` → `main()` się wykona
- `import plik` (np. w testach) → `__name__ == "plik"` → `main()` się **nie** wykona
- Dzięki temu ten sam plik jest i **programem**, i **modułem** z funkcjami

CZĘŚĆ II

Praktyka + blok AI

Blok AI: zasady

1. **Najpierw Ty.** AI dopiero po zrobieniu zadań ★
2. **AI wyjaśnia, Ty weryfikujesz.** Każde stwierdzenie sprawdź w kodzie lub dokumentacji
3. **Każda zmiana → testy.** Refaktoryzacja nie może zmienić zachowania
4. **Kod, którego nie umiesz wyjaśnić, nie jest Twój**
5. Wszystko zapisz w `AI.md`: prompt, decyzje, co najmniej jeden błąd AI

„AI nie zastępuje myślenia” – wytyczne Instytutu Informatyki ALK

Blok AI: przykładowe prompty

- *„Wyjaśnij linijka po linijce, co robi ta funkcja i dla jakich danych może zadziałać źle: ...”*
- *„Zaproponuj refaktoryzację zgodną z PEP 8. Nie zmieniaj nazw funkcji ani ich zachowania. Każdą zmianę uzasadnij.”*
- *„Podaj 5 przypadków brzegowych dla funkcji is_palindrome.”*

Na co uważać:

- AI potrafi zmienić zachowanie „przy okazji” (np. inne traktowanie polskich liter)
- AI potrafi pewnym tonem wymyślić nieistniejącą funkcję lub parametr
- Krótsze ≠ czytelniejsze

Ciekawostki

- **Podprogram** (*subroutine*) wynalazł m.in. **David Wheeler** dla komputera EDSAC w Cambridge (ok. 1949–1951) – skok do podprogramu nazywano „skokiem Wheelera”
- Słowo **lambda** pochodzi z **rachunku lambda** Alonzo Churcha (lata 30. XX w.) – matematycznego modelu obliczeń opartego wyłącznie na funkcjach
- Guido van Rossum świadomie **nie dodał** do Pythona optymalizacji rekurencji ogonowej – pełny stos wywołań ułatwia debugowanie (blog *Neopythonic*, 2009)

Dzisiejsze zadania

PLIK	ZADANIE
★ <code>zad1_kalkulator.py</code>	kalkulator funkcyjny
★ <code>zad2_napisy.py</code>	przetwarzanie napisów
★ <code>zad3_csv.py</code>	analiza CSV
★ <code>zad4_rekurencja.py</code>	rekurencja
 <code>AI.md</code>	wyjaśnienie i refaktoryzacja z AI
☆ <code>zad5_lambda.py</code>	lambdy i zasięg

Sprawdzenie i oddanie

```
cd lab07_funkcje
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 07"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
-  **Kolokwium I: jutro, środa 02.12, 14:00–17:15, sala A/136** – zakres: laby 01–07

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 8
- M. Dawson, *Python dla każdego*, Helion 2015 – rozdz. 6
- PEP 257 – *Docstring Conventions*
- Dokumentacja: *Defining Functions* (docs.python.org/3/tutorial/controlflow.html)

Następne zajęcia (02.12):  Kolokwium I